

```

import React, { forwardRef, useRef, useCallback } from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface TextBoxInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface TextBoxActionButtonConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>, value: string) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface TextBoxProps extends React.InputHTMLAttributes<HTMLInputElement> {
  label?: string;
  error?: string;
  innerIcons?: TextBoxInnerIconConfig[];
  actionButtons?: TextBoxActionButtonConfig[];
  prefixIcon?: LucideIcon;
  suffixIcon?: LucideIcon;
  prefixText?: string;
  suffixText?: string;
}

export const TextBox = forwardRef<HTMLInputElement, TextBoxProps>(
  (
    {
      label,
      error,
      innerIcons = [],
      actionButtons = [],
      prefixIcon,
      suffixIcon,
      prefixText,
      suffixText,
      className,
      disabled,
      required,
      placeholder,
      type = "text",
      ...props
    },
    ref
  )

```

```

    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const hasError = Boolean(error);

    const handleRef = useCallback(
      (node: HTMLInputElement | null) => {
        inputRef.current = node;
        if (ref) {
          if (typeof ref === "function") {
            ref(node);
          } else {
            ref.current = node;
          }
        }
      },
      [ref],
    );

    const limitedInnerIcons = innerIcons.slice(0, 4);
    const limitedActionButtons = actionButtons.slice(0, 4);

    return (
      <div className={cn("w-full", className)}>
        {label && (
          <label
            className={cn(
              "block text-sm font-medium text-text-main mb-1.5",
              disabled && "opacity-50",
            )}
          >
            {label}
            {required && (
              <span className="text-destructive ml-0.5" aria-hidden="true">
                *
              </span>
            )}
          </label>
        )}
        <div className="flex items-center w-full gap-2">
          <div
            className={cn(
              "relative flex items-center flex-1 min-w-0",
              "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
              "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
              "rounded-surface",
              "text-text-main placeholder:text-muted-foreground/60",
              "focus-within:ring-2 focus-within:ring-amber-500/20",
              "focus-within:border-amber-500 focus-within:bg-amber-500/5",
              "disabled:opacity-50 disabled:pointer-events-none"
            )}
          >

```

```

disabled:cursor-not-allowed",
    "transition-all duration-200 ease-out",
    hasError &&
    "border-destructive/50 focus-within:ring-destructive/50
focus-within:border-destructive/50",
    disabled && "bg-muted/50",
  )}
>
  {(prefixIcon || prefixText) && (
    <div
      className={cn(
        "flex items-center justify-center px-3",
        "text-muted-foreground/60",
        disabled && "opacity-50",
      )}
      aria-hidden="true"
    >
      {prefixIcon && (
        <span className="w-4 h-4" aria-hidden="true">
          {React.createElement(prefixIcon, { className: "w-4 h-4" })}
        </span>
      )}
      {prefixText && (
        <span className="text-sm font-medium">{prefixText}</span>
      )}
    </div>
  )}
  <input
    ref={handleRef}
    type={type}
    className={cn(
      "flex-1 bg-transparent border-none outline-none",
      "px-4 py-2.5",
      "text-text-main placeholder:text-muted-foreground/60",
      "disabled:opacity-50 disabled:pointer-events-none
disabled:cursor-not-allowed",
      "w-full min-w-0",
      prefixIcon || prefixText ? "pl-0" : "pl-4",
      suffixIcon || suffixText || limitedInnerIcons.length > 0
        ? "pr-0"
        : "pr-4",
    )}
    disabled={disabled}
    required={required}
    placeholder={placeholder}
    onChange={(e) => props.onChange?.(e)}
    {...props}
  />
  {(suffixIcon || suffixText) && (
    <div

```

```

        className={cn(
          "flex items-center justify-center px-3",
          "text-muted-foreground/60",
          disabled && "opacity-50",
        )}
        aria-hidden="true"
      >
      {suffixIcon && (
        <span className="w-4 h-4" aria-hidden="true">
          {React.createElement(suffixIcon, { className: "w-4 h-4" })}
        </span>
      )}
      {suffixText && (
        <span className="text-sm font-medium">{suffixText}</span>
      )}
    </div>
  )}
  {limitedInnerIcons.length > 0 && (
    <div className="absolute right-3 flex items-center gap-1">
      {limitedInnerIcons.map((iconConfig, index) => (
        <span
          key={index}
          className={cn(
            "flex items-center justify-center",
            "w-5 h-5",
            "text-muted-foreground/60",
            iconConfig.className,
          )}
          aria-hidden="true"
        >
          <iconConfig.icon className="w-4 h-4" aria-hidden="true" />
        </span>
      )})}
    </div>
  )}
</div>
{limitedActionButtonButtons.length > 0 && (
  <div className="flex items-center gap-1.5 shrink-0">
    {limitedActionButtonButtons.map((action, index) => (
      <button
        key={index}
        type="button"
        className={cn(
          "relative flex items-center justify-center",
          "w-9 h-9 rounded-md",
          "text-muted-foreground/60 hover:text-foreground",
          "bg-transparent hover:bg-accent",
          "transition-transform duration-100",
          "active:scale-95",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring

```

```

focus-visible:ring-offset-2",
      "disabled:opacity-50 disabled:pointer-events-none",
    )}
    aria-label={action.tooltipText}
    disabled={disabled || action.disabled}
    onClick={(e) =>
      action.onClick(e, inputRef.current?.value || "")
    }
  >
    <action.icon className="w-4 h-4" aria-hidden="true" />
  </button>
  )}
</div>
)}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
);
},
);

```

```

TextBox.displayName = "TextBox";

```